

Traduction des instructions

Sommaire

LValue

Structures de contrôle

Une différence entre votre projet et le C

Expressions et instructions

```

Exp : Exp OR TB
    | TB ;
TB : TB AND FB
    | FB ;
(...)
E : E ADDSUB T
    | T ;
T : T DIVSTAR F
    | F ;
F : ADDSUB F
    | '!' F
    | '(' Exp ')'
    | NUM
    | CHARACTER
    | IDENT
    | IDENT '(' Arguments ')' ;
    
```

```

Instr: IDENT '=' Exp ';'
    | IF '(' Exp ')' Instr
    | IF '(' Exp ')' Instr ELSE Instr
    | WHILE '(' Exp ')' Instr
    | IDENT '(' Arguments ')' ';'
    | RETURN Exp ';'
    | RETURN ';'
    | '{ SuiteInstr }'
    | ';' ;
    
```

Expressions

$x*x+4$

$x==y$

$\text{sqrt}(x*x+4)$

Instructions

$y=0;$

`return t;`

`printf("%s\n", msg);`

En C, toute expression suivie par un ";" est une
instruction

$y=0$

$1+2$

$y=0;$

$1+2;$

Pas dans le projet

LValue

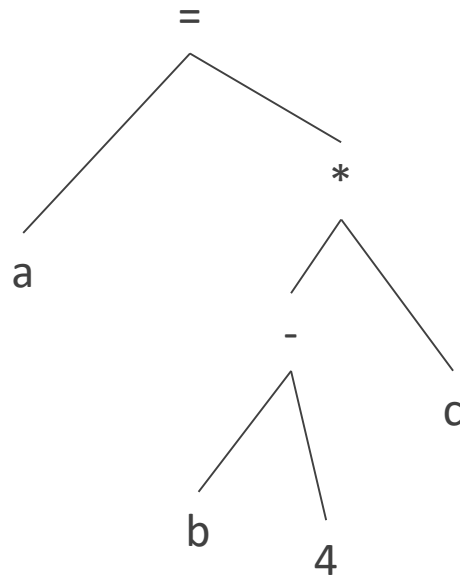
```
Instr:  LValue '=' Exp ';'
(... )
F :  ADDSUB F
    |  '!' F
    |  IDENT '(' Arguments ')'
    |  '(' Exp ')'
    |  LValue
    |  NUM
    |  CHARACTER
    ;
LValue:  IDENT ;
```

Une expression qui a une adresse

Une LValue peut apparaitre à gauche dans une affectation

- variable **size=2;**
- élément de tableau **size[i]=2;**
- champ de structure **array.size=2;**
- déréférencement de pointeur ***size=2;**

Traduction des affectations (*assignments*)



`tmp1 := $a`

`tmp2 := $4`

`tmp3 := b - tmp2`

`tmp4 := tmp3 * c`

`base[tmp1] := tmp4`

`tmp1 := $4`

`tmp2 := b - tmp1`

`tmp3 := tmp2 * c`

`a := tmp3`

Simple pour une variable,
non pour un champ de structure

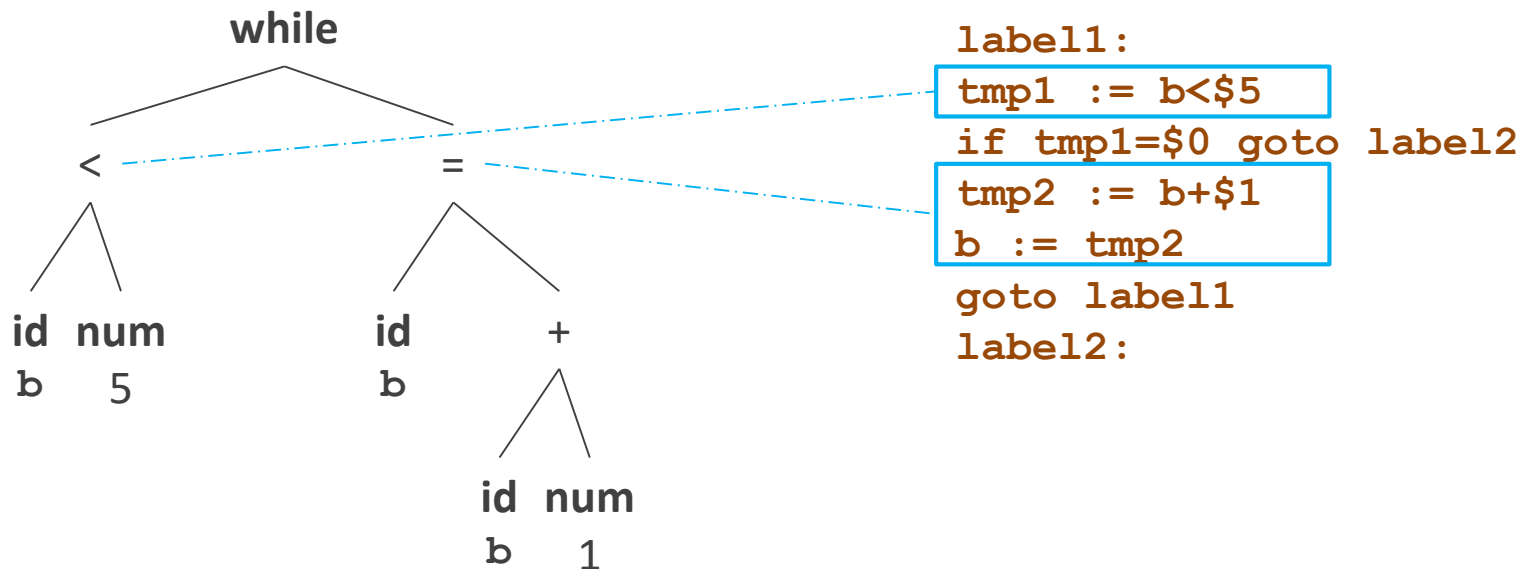
Sommaire

[LValue](#)

Structures de contrôle

Traduction des structures de contrôle

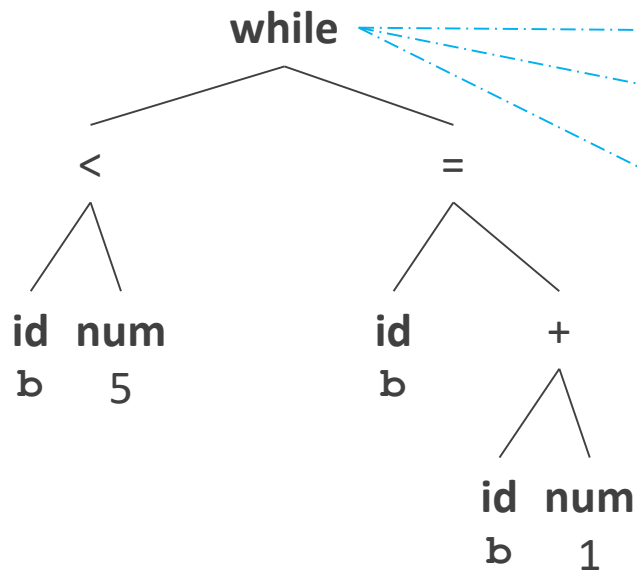
```
while (b<5) b=b+1;
```



L'arbre abstrait ne contient pas explicitement de sauts

Traduction des structures de contrôle

```
while (b<5) b=b+1;
```



label1:

tmp1 := b<\$5

if tmp1=\$0 goto label2

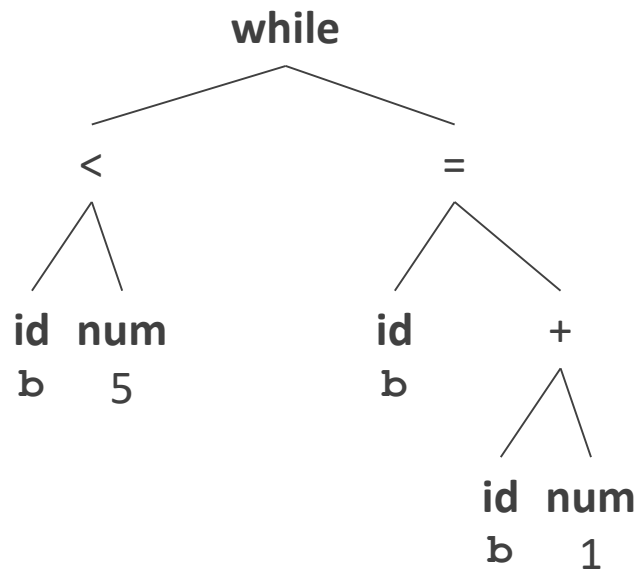
tmp2 := b+\$1

b := tmp2

goto label1

label2:

Traduction des structures de contrôle



```

label1:
tmp1 := b<$5
if tmp1=$0 goto label2
tmp2 := b+$1
b := tmp2
goto label1
label2:
  
```

Nœud
while

```

begin = newlabel() ; after = newlabel() ; write(begin ':' );
visit(while.firstchild);
write('if' while.firstchild.place '= $0 goto' after);
visit(while.secondchild);
write('goto' begin); write(after ':' );
  
```

Merci

CONTACT

ERIC LAPORTE

+33 1 60 95 75 52

ERIC.LAPORTE@UNIV-EIFFEL.FR